Implied Volatility Surface & Options Analysis

A 7-day project roadmap — theory, math and notebook code

HEM Bellyday (Day)
Financial Engineering, Institute of Technology of Cambodia

October 2026

How to use this guide. Everything lives in **one Jupyter notebook**, `iv_surface.ipynb`, organised by day. Each day has: a goal, the theory (copy the boxed definitions into Markdown cells), numbered code cells (*Cell 3.2* = Day 3, cell 2), and a task checklist. Run cells top to bottom. The only `.py` file in the project is the Streamlit app on Day 7, because Streamlit cannot run notebooks.

Contents

1 Project overview **3**

2 What you will know at the end **4**

3 Setup and repo structure **4**

4 Day 1 — Option basics and the data pipeline **6**

4.1 Theory . . . 6

4.2 Code . . . 6

4.3 Tasks . . . 8

5 Day 2 — Black–Scholes pricing and Greeks **9**

5.1 Theory . . . 9

5.2 Code . . . 10

5.3 Tasks . . . 11

6 Day 3 — Implied forward and implied volatility **12**

6.1 Theory . . . 12

6.2 Code . . . 12

6.3 Tasks . . . 14

7 Day 4 — Smile, skew and term structure **15**

7.1 Theory . . . 15

7.2 Code . . . 15

7.3 Tasks . . . 16

8 Day 5 — The surface: SVI fit and arbitrage checks **17**

8.1 Theory . . . 17

8.2 Code . . . 17

8.3 Tasks . . . 19

9 Day 6 — Options analysis with the surface **20**

9.1 Theory . . . 20

9.2 Code . . . 20

9.3 Tasks . . . 22

10 Day 7 — Streamlit app, README and GitHub **23**

10.1 Deploy . . . 24

10.2 README checklist . . . 24

10.3 Final push . . . 24

11 Stretch goals and further reading **25**

1 Project overview

In 7 days you build a notebook that downloads real option prices from `yfinance`, backs out implied volatility for every contract, fits a smooth arbitrage-checked volatility surface, and uses it for pricing, risk and probability analysis — then publish it on GitHub with a Streamlit app.

Description

An option price on its own is hard to compare across strikes and expiries. **Implied volatility** (IV) turns every price into one common number: the volatility that makes Black–Scholes match the market. Plot IV against strike and maturity and you get the **implied volatility surface** — the main object option traders and quants look at every day.

The problem

- **Black–Scholes assumes one constant volatility.** Real markets show a different IV at each strike and maturity (smile and skew). The model is "wrong", yet everyone still quotes prices in it.
- **Market data is discrete and noisy.** You only see some strikes and expiries, with bid–ask noise and stale quotes. To price an option that is not listed, you need a continuous surface.
- **Naive interpolation creates arbitrage.** A badly smoothed surface can imply negative probabilities or calendar arbitrage, which breaks every model built on top of it.

Why this matters for quantitative finance

- **It is the starting point of every derivatives desk.** Market makers, volatility traders and risk teams mark books, hedge and price exotics off the IV surface.
- **It connects theory to real data.** Stochastic calculus (Black–Scholes), numerical methods (root-finding, optimisation, interpolation) and statistics (fitting, error analysis) on a real, messy dataset — exactly what quant interviews test.
- **The surface contains market expectations.** Skew shows crash fear, the term structure shows expected future volatility, and the second derivative of call prices gives the market's risk-neutral probability distribution.
- **It is a strong portfolio project.** A clean notebook, a live app and a clear README show you can take a quant idea from math to working product.

The week at a glance

Day	Focus	Deliverable at end of day
1	Option basics, data	Clean option chain saved to `data/`, first commit
2	Black–Scholes, Greeks	Tested pricer + Greeks, Monte Carlo check
3	Implied volatility	Implied forward, IV for every option, solver tests
4	Smile, skew, term structure	Smile plots, skew metrics, risk reversal table
5	Surface: SVI + arbitrage	Fitted 3D surface, arbitrage report, exported params
6	Options analysis	Risk-neutral density, VRP, strategy Greeks
7	App + GitHub	Streamlit app online, README, final push

Plan about 4–6 hours per day. If a day runs long, finish its tasks the next morning before starting the new day — every day builds on the one before.

2 What you will know at the end

Area	You will be able to...
Option theory	Explain payoffs, put–call parity, no-arbitrage bounds, risk-neutral pricing
Stochastic calculus	Derive the Black–Scholes formula from GBM under $\mathbb{Q}$; state the BS PDE
Greeks	Derive and code Delta, Gamma, Vega, Theta, Rho; verify with finite differences
Numerical methods	Implement Newton–Raphson and Brent; handle edge cases and convergence
Volatility	Read smiles, skew and term structure; explain why skew exists
Surface modelling	Fit SVI, interpolate in total variance, test butterfly and calendar arbitrage
Probability	Extract the risk-neutral density (Breeden–Litzenberger) and check it is valid
Trading insight	Measure the volatility risk premium; analyse straddles, risk reversals, butterflies
Engineering	Build a clean notebook, a Streamlit app, a GitHub repo with README and results

Interview questions you should answer easily afterwards

1. Why is vega always positive, and why does that make implied vol unique?
2. Why is equity index skew downward-sloping?
3. Why do we interpolate total variance $\sigma^2 T$ instead of σ?
4. What does $g(k) < 0$ (butterfly arbitrage) mean economically?
5. How do you recover the market-implied probability distribution from option prices?
6. Why do index option sellers earn a premium on average?

3 Setup and repo structure

Terminal

```
mkdir iv-surface && cd iv-surface
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install yfinance numpy pandas scipy plotly pyarrow streamlit jupyter
pip freeze > requirements.txt
git init
```

Repo layout (only one .py file)

```
iv-surface/
|-- README.md
|-- requirements.txt
|-- iv_surface.ipynb      <- Days 1-6, all theory + code
|-- app.py                <- Day 7, Streamlit app (reads data/ exported by the notebook)
|-- data/                 <- raw chains, IV points, SVI params (exported by the notebook)
|-- figures/              <- saved plots for the README
```

Notebook layout. Use one Markdown heading per day (`# Day 1 -- Data`, …) and put the theory boxes from this guide in Markdown cells above the code. Jupyter renders `$...$` and `$$...$$` as math, so you can paste the formulas directly. Commit at the end of each day: a daily commit history looks good on GitHub.

Two notebook rules that avoid bugs:

- Always write `df["T"]`, never `df.T` — in pandas `.T` is the *transpose*.
- "Restart kernel & run all" before every commit, so the notebook really runs top to bottom.

4 Day 1 — Option basics and the data pipeline

Goal: a clean DataFrame of option quotes (strike, maturity, mid price, type, spot) saved to disk, ready for pricing.

4.1 Theory

Definition — European call and put

A call gives the right to *buy* the underlying at strike K at maturity T; a put gives the right to *sell*. Payoffs:

$$C_T = (S_T - K)^+, \qquad P_T = (K - S_T)^+.$$

Definition — Forward and moneyness

With risk-free rate r and dividend yield q, the forward price is $F = S\,e^{(r-q)T}$. Log-moneyness is

$$k = \ln \frac{K}{F}.$$

$k < 0$: the call is in-the-money (put out-of-the-money). $k = 0$: at-the-money forward.

Proposition — No-arbitrage bounds

$$\max\left(Se^{-qT} - Ke^{-rT}, 0\right) \le C \le Se^{-qT}, \qquad \max\left(Ke^{-rT} - Se^{-qT}, 0\right) \le P \le Ke^{-rT}.$$

Theorem — Put–call parity (model-free)

$$C - P = Se^{-qT} - Ke^{-rT}.$$

Proof. Long call + short put pays $(S_T - K)^+ - (K - S_T)^+ = S_T - K$ in every state. A portfolio of e^{-qT} shares and a loan of Ke^{-rT} pays the same. Equal payoffs ⇒ equal prices, otherwise there is arbitrage. ■

Why keep only OTM options. OTM options are the most liquid and their whole price is time value, so the IV is accurate. ITM options are mostly intrinsic value and small quote errors become large IV errors. Rule: puts for $K < S$, calls for $K \ge S$.

4.2 Code

Cell 1.1: Imports and settings

```
import os, json, glob, warnings
import numpy as np
import pandas as pd
import yfinance as yf
import plotly.graph_objects as go
from scipy.stats import norm
from scipy.optimize import brentq, minimize
from scipy.interpolate import griddata

warnings.filterwarnings("ignore")
os.makedirs("data", exist_ok=True)
```

```
os.makedirs("figures", exist_ok=True)

TICKER = "SPY"        # change to "AAPL" etc. and re-run the whole notebook
MAX_EXPIRIES = 10
```

Cell 1.2: Download functions

```
def get_risk_free_rate():
    """13-week US T-bill yield (^IRX), as a decimal."""
    irx = yf.Ticker("^IRX").history(period="5d")["Close"].iloc[-1]
    return float(irx) / 100.0

def fetch_chain(ticker="SPY", max_expiries=10):
    tk = yf.Ticker(ticker)
    spot = float(tk.history(period="1d")["Close"].iloc[-1])
    today = pd.Timestamp.today().normalize()
    frames = []
    for exp in tk.options[:max_expiries]:
        chain = tk.option_chain(exp)
        for d, kind in [(chain.calls, "call"), (chain.puts, "put")]:
            d = d[["strike", "bid", "ask", "lastPrice", "volume",
                   "openInterest", "impliedVolatility"]].copy()
            d["type"] = kind
            d["expiry"] = pd.to_datetime(exp)
            frames.append(d)
    df = pd.concat(frames, ignore_index=True)
    df["spot"] = spot
    df["T"] = (df["expiry"] - today).dt.days / 365.0
    df["mid"] = (df["bid"] + df["ask"]) / 2
    return df.rename(columns={"impliedVolatility": "iv_yahoo"})
```

Cell 1.3: Download and save (raw data is kept so results are reproducible)

```
r = get_risk_free_rate()
raw = fetch_chain(TICKER, MAX_EXPIRIES)
S0 = raw["spot"].iloc[0]
raw.to_parquet(f"data/raw_{TICKER}_{pd.Timestamp.today():%Y%m%d}.parquet")
print(f"r = {r:.4f}   spot = {S0:.2f}   rows = {len(raw)}   expiries = {raw['expiry'].nunique()}")

# Later, to work offline on the same snapshot:
# raw = pd.read_parquet(sorted(glob.glob(f"data/raw_{TICKER}_*.parquet"))[-1])
```

Cell 1.4: Cleaning, printing what each filter removes

```
def clean_chain(df, min_T=7/365, m_low=0.7, m_high=1.3, max_rel_spread=0.5, verbose=True):
    steps = [
        ("bid > 0 and ask > bid", lambda d: d[(d["bid"] > 0) & (d["ask"] > d["bid"])]),
        ("relative spread",       lambda d: d[(d["ask"] - d["bid"]) / d["mid"] < max_rel_spread]),
        ("min maturity",          lambda d: d[d["T"] > min_T]),
        ("moneyness range",       lambda d: d[(d["strike"] / d["spot"]).between(m_low, m_high)]),
        ("OTM only",              lambda d: d[((d["type"] == "call") & (d["strike"] >= d["spot"])) |
                                              ((d["type"] == "put") & (d["strike"] < d["spot"]))]),
        ("open interest > 0",     lambda d: d[d["openInterest"].fillna(0) > 0]),
    ]
    d = df.copy()
    for name, f in steps:
        n0 = len(d)
        d = f(d)
        if verbose:
            print(f"{name:24s} removed {n0 - len(d):5d}   left {len(d):5d}")
    return d.reset_index(drop=True)

df = clean_chain(raw)
```

Cell 1.5: No-arbitrage bound check and a first plot

```
def bound_violations(d, r, q=0.0):
    S, K, T = d["spot"], d["strike"], d["T"]
    disc_r, disc_q = np.exp(-r * T), np.exp(-q * T)
    is_call = d["type"] == "call"
    lower = np.where(is_call, np.maximum(S*disc_q - K*disc_r, 0), np.maximum(K*disc_r - S*disc_q, 0))
    upper = np.where(is_call, S*disc_q, K*disc_r)
    return int(((d["mid"] <= lower) | (d["mid"] >= upper)).sum())

print("violations (raw):  ", bound_violations(raw[raw["bid"] > 0], r))
print("violations (clean):", bound_violations(df, r))

fig = go.Figure()
for exp in sorted(raw["expiry"].unique())[:3]:
    for kind in ["call", "put"]:
        g = raw[(raw["expiry"] == exp) & (raw["type"] == kind) & (raw["bid"] > 0)]
        fig.add_scatter(x=g["strike"], y=g["mid"], mode="lines", name=f"{kind} {exp:%Y-%m-%d}")
fig.update_layout(title="Mid price vs strike", xaxis_title="Strike", yaxis_title="Mid price")
fig.show()
```

4.3 Tasks

☐ Create the GitHub repo, the notebook and the folder layout.

☐ Download SPY and one single stock (e.g. AAPL); save both raw files.

☐ Read the cleaning printout: which filter removes the most rows, and why?

☐ Check in the plot that calls fall and puts rise as K increases.

☐ Write put–call parity and its proof in a Markdown cell.

☐ Commit: `git add . && git commit -m "Day 1: data pipeline"`.

Timing note. yfinance quotes are delayed and can be stale outside US market hours. Phnom Penh is 11–12 hours ahead of New York, so download during US trading hours (about 20:30–03:00 Phnom Penh time), or on the weekend use the last saved file.

5 Day 2 — Black–Scholes pricing and Greeks

Goal: a vectorised, tested Black–Scholes pricer with all Greeks — the engine every later day uses.

5.1 Theory

Model — Geometric Brownian motion under $\mathbb{Q}$

$$dS_t = (r-q)\,S_t\,dt + \sigma S_t\,dW_t^{\mathbb{Q}}.$$

Applying Itô's lemma to $\ln S_t$ gives $d\ln S_t = (r-q-\tfrac{1}{2}\sigma^2)dt + \sigma dW_t$, so

$$S_T = S_0 \exp\Big((r-q-\tfrac{1}{2}\sigma^2)T + \sigma\sqrt{T}\,Z\Big), \qquad Z\sim\mathcal{N}(0,1).$$

Theorem — Black–Scholes formula

With risk-neutral pricing $C = e^{-rT}\,\mathbb{E}^{\mathbb{Q}}[(S_T-K)^+]$:

$$d_1 = \frac{\ln(S/K) + (r-q+\tfrac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T},$$

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2), \qquad P = Ke^{-rT}N(-d_2) - Se^{-qT}N(-d_1).$$

Proof sketch. Write $\mathbb{E}[(S_T-K)^+] = \mathbb{E}[S_T\mathbf{1}_{S_T>K}] - K\,\mathbb{Q}(S_T>K)$. The second term is $KN(d_2)$ since $S_T > K \iff Z > -d_2$. For the first, change measure to the one with numeraire S (or complete the square in the Gaussian integral): it equals $S_0 e^{(r-q)T}N(d_1)$. Multiply by e^{-rT}. ■

Black–Scholes PDE

Delta-hedging the option with the stock removes all risk, so the hedged portfolio must earn r:

$$\frac{\partial V}{\partial t} + \tfrac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2} + (r-q)S\frac{\partial V}{\partial S} - rV = 0.$$

Greeks (φ = standard normal density):

Greek	Call	Put
Delta $\partial V/\partial S$	$e^{-qT}N(d_1)$	$-e^{-qT}N(-d_1)$
Gamma $\partial^2 V/\partial S^2$	$\dfrac{e^{-qT}\varphi(d_1)}{S\sigma\sqrt{T}}$	same
Vega $\partial V/\partial\sigma$	$Se^{-qT}\varphi(d_1)\sqrt{T}$	same
Theta $\partial V/\partial t$	$-\dfrac{Se^{-qT}\varphi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2) + qSe^{-qT}N(d_1)$	$-\dfrac{Se^{-qT}\varphi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(-d_2) - qSe^{-qT}N(-d_1)$
Rho $\partial V/\partial r$	$KTe^{-rT}N(d_2)$	$-KTe^{-rT}N(-d_2)$

Key fact for Day 3: Vega $= Se^{-qT}\varphi(d_1)\sqrt{T} > 0$ for every $\sigma > 0$, so the option price is strictly increasing in σ.

5.2 Code

Cell 2.1: Pricer and Greeks (work with numbers, arrays or pandas columns)

```
def _arr(*xs):
    return [np.asarray(x, dtype=float) for x in xs]

def bs_d1_d2(S, K, T, r, q, sigma):
    S, K, T, sigma = _arr(S, K, T, sigma)
    vsqrt = sigma * np.sqrt(T)
    d1 = (np.log(S / K) + (r - q + 0.5 * sigma**2) * T) / vsqrt
    return d1, d1 - vsqrt

def bs_price(S, K, T, r, q, sigma, kind="call"):
    d1, d2 = bs_d1_d2(S, K, T, r, q, sigma)
    S, K, T = _arr(S, K, T)
    disc_r, disc_q = np.exp(-r * T), np.exp(-q * T)
    call = S * disc_q * norm.cdf(d1) - K * disc_r * norm.cdf(d2)
    put = K * disc_r * norm.cdf(-d2) - S * disc_q * norm.cdf(-d1)
    return np.where(np.asarray(kind) == "call", call, put)

def bs_greeks(S, K, T, r, q, sigma, kind="call"):
    d1, d2 = bs_d1_d2(S, K, T, r, q, sigma)
    S, K, T, sigma = _arr(S, K, T, sigma)
    disc_r, disc_q = np.exp(-r * T), np.exp(-q * T)
    pdf, sqT = norm.pdf(d1), np.sqrt(T)
    is_call = np.asarray(kind) == "call"
    delta = np.where(is_call, disc_q * norm.cdf(d1), -disc_q * norm.cdf(-d1))
    gamma = disc_q * pdf / (S * sigma * sqT)
    vega = S * disc_q * pdf * sqT
    decay = -S * disc_q * pdf * sigma / (2 * sqT)
    theta = np.where(is_call,
                     decay - r*K*disc_r*norm.cdf(d2) + q*S*disc_q*norm.cdf(d1),
                     decay + r*K*disc_r*norm.cdf(-d2) - q*S*disc_q*norm.cdf(-d1))
    rho = np.where(is_call, K*T*disc_r*norm.cdf(d2), -K*T*disc_r*norm.cdf(-d2))
    return {"delta": delta, "gamma": gamma, "vega": vega, "theta": theta, "rho": rho}
```

Cell 2.2: Tests in a cell: put–call parity and finite-difference Greeks

```
tS, tK, tT, tr, tq, ts = 100.0, 105.0, 0.5, 0.04, 0.01, 0.2
price = lambda S=tS, K=tK, T=tT, r=tr, s=ts, kind="call": float(bs_price(S, K, T, r, tq, s, kind))
G = {k: float(v) for k, v in bs_greeks(tS, tK, tT, tr, tq, ts).items()}
h = 1e-4

assert np.isclose(price() - price(kind="put"), tS*np.exp(-tq*tT) - tK*np.exp(-tr*tT))
assert np.isclose((price(S=tS+h) - price(S=tS-h)) / (2*h), G["delta"], atol=1e-6)
assert np.isclose((price(S=tS+h) - 2*price() + price(S=tS-h)) / h**2, G["gamma"], atol=1e-4)
assert np.isclose((price(s=ts+h) - price(s=ts-h)) / (2*h), G["vega"], atol=1e-5)
assert np.isclose(-(price(T=tT+h) - price(T=tT-h)) / (2*h), G["theta"], atol=1e-5)
assert np.isclose((price(r=tr+h) - price(r=tr-h)) / (2*h), G["rho"], atol=1e-5)
print("All Black-Scholes tests passed")
```

Theta is $\partial V/\partial t$ (calendar time), and t moving forward means T shrinking — that is why the finite difference in T has a minus sign.

Cell 2.3: Monte Carlo check: error should shrink like $1/\sqrt{n}$

```
exact = price()
rng = np.random.default_rng(42)
for n in [10**3, 10**4, 10**5, 10**6]:
    Z = rng.standard_normal(n)
    ST = tS * np.exp((tr - tq - 0.5*ts**2)*tT + ts*np.sqrt(tT)*Z)
    mc = np.exp(-tr*tT) * np.maximum(ST - tK, 0).mean()
    print(f"n = {n:>8d}   MC = {mc:.4f}   exact = {exact:.4f}   error = {abs(mc - exact):.4f}")
```

Cell 2.4: Delta and Gamma vs spot for three maturities

```
S_grid = np.linspace(70, 130, 200)
fig = go.Figure()
for T_ in [0.05, 0.25, 1.0]:
    g = bs_greeks(S_grid, 100, T_, 0.04, 0.0, 0.2)
    fig.add_scatter(x=S_grid, y=g["gamma"], name=f"Gamma T={T_}")
fig.update_layout(title="Gamma spikes near ATM as maturity shrinks", xaxis_title="Spot")
fig.show()
```

5.3 Tasks

☐ Write the full derivation of C in a Markdown cell (the two Gaussian integrals).

☐ All asserts pass; Monte Carlo error shrinks by about $\sqrt{10} \approx 3.2$ per row.

☐ Add a Delta plot next to the Gamma plot; explain the shape in two sentences.

☐ Commit: `"Day 2: Black-Scholes and Greeks"`.

6 Day 3 — Implied forward and implied volatility

Goal: read the forward and dividend yield from the market, then turn every cleaned price into an implied volatility with a fast, robust solver.

6.1 Theory

Implied forward from put–call parity

Rewriting parity with $F = Se^{(r-q)T}$:

$$C(K) - P(K) = e^{-rT}(F - K) \quad \Longrightarrow \quad F = K + e^{rT}\big(C(K) - P(K)\big).$$

Take the median over the 5 strikes nearest to spot, then the implied dividend yield is $q = r - \frac{1}{T}\ln\frac{F}{S}$. This avoids guessing q and absorbs dividends and borrow costs.

Definition — Implied volatility

The implied volatility of an option with market price V_{mkt} is the σ_{imp} solving

$$f(\sigma) = \text{BS}(S, K, T, r, q, \sigma) - V_{\text{mkt}} = 0.$$

Theorem — Existence and uniqueness

If V_{mkt} lies strictly inside the no-arbitrage bounds of Day 1, there is exactly one $\sigma_{\text{imp}} > 0$.
Proof. BS is continuous in σ. As $\sigma \to 0$ the price tends to the lower bound; as $\sigma \to \infty$ it tends to the upper bound. So f changes sign, and by the intermediate value theorem a root exists. Since $f'(\sigma) = \text{Vega} > 0$, f is strictly increasing, so the root is unique. ■

Newton–Raphson uses vega as the derivative and converges quadratically near the root:

$$\sigma_{n+1} = \sigma_n - \frac{\text{BS}(\sigma_n) - V_{\text{mkt}}}{\text{Vega}(\sigma_n)}.$$

It fails when vega is tiny (deep OTM, very short T): the step explodes. **Initial guess** (Brenner–Subrahmanyam, from a Taylor expansion near ATM): $\sigma_0 \approx \sqrt{2\pi/T}\, V_{\text{mkt}}/S$.

Brent's method brackets the root in $[10^{-4}, 5]$ and always converges, but more slowly. Production rule: *Newton first, Brent as fallback.*

6.2 Code

Cell 3.1: Implied forward and dividend yield per expiry

```
def implied_forward(raw, r, n_near=5):
    rows = []
    quotes = raw[(raw["bid"] > 0) & (raw["ask"] > raw["bid"])]
    for exp, g in quotes.groupby("expiry"):
        c = g[g["type"] == "call"].set_index("strike")["mid"]
        p = g[g["type"] == "put"].set_index("strike")["mid"]
        common = c.index.intersection(p.index)
        if len(common) == 0:
            continue
        S, T = g["spot"].iloc[0], g["T"].iloc[0]
        near = sorted(common, key=lambda K: abs(K - S))[:n_near]
```

```
        F = float(np.median([K + np.exp(r*T) * (c[K] - p[K]) for K in near]))
        rows.append({"expiry": exp, "T": T, "F": F, "q": r - np.log(F / S) / T})
    return pd.DataFrame(rows).set_index("expiry")

fwd = implied_forward(raw, r)
df["F"] = df["expiry"].map(fwd["F"])
df["q"] = df["expiry"].map(fwd["q"])
df = df.dropna(subset=["F"])
fwd
```

Cell 3.2: Implied volatility solver: Newton with Brent fallback

```
def implied_vol(price, S, K, T, r, q, kind="call", tol=1e-8, max_iter=50):
    disc_r, disc_q = np.exp(-r*T), np.exp(-q*T)
    if kind == "call":
        lower, upper = max(S*disc_q - K*disc_r, 0.0), S*disc_q
    else:
        lower, upper = max(K*disc_r - S*disc_q, 0.0), K*disc_r
    if not (lower < price < upper):
        return np.nan                                   # outside no-arbitrage bounds
    if price - lower < 1e-8:
        return np.nan                                   # no time value left: IV not identifiable

    sigma = float(np.clip(np.sqrt(2*np.pi/T) * price / S, 0.05, 2.0))
    for _ in range(max_iter):                           # Newton-Raphson
        diff = float(bs_price(S, K, T, r, q, sigma, kind)) - price
        if abs(diff) < tol:
            return sigma
        vega = float(bs_greeks(S, K, T, r, q, sigma, kind)["vega"])
        if vega < 1e-8:
            break
        sigma -= diff / vega
        if not (1e-4 < sigma < 5.0):
            break

    f = lambda s: float(bs_price(S, K, T, r, q, s, kind)) - price
    try:                                                # Brent fallback
        return brentq(f, 1e-4, 5.0, xtol=tol)
    except ValueError:
        return np.nan

df["iv"] = [implied_vol(x.mid, x.spot, x.strike, x["T"], r, x.q, x.type)
            for _, x in df.iterrows()]
print(f"IV failed for {df['iv'].isna().sum()} of {len(df)} options")
df = df.dropna(subset=["iv"]).reset_index(drop=True)
```

Cell 3.3: Round-trip test over a grid of strikes, maturities and vols

```
errors = []
for K_ in [60, 80, 95, 100, 105, 120, 150]:
    for T_ in [0.02, 0.1, 0.5, 2.0]:
        for s_ in [0.08, 0.2, 0.5, 1.0]:
            kind = "put" if K_ < 100 else "call"
            p_ = float(bs_price(100, K_, T_, 0.04, 0.01, s_, kind))
            iv_ = implied_vol(p_, 100, K_, T_, 0.04, 0.01, kind)
            errors.append({"K": K_, "T": T_, "sigma": s_, "error": abs(iv_ - s_)})
errors = pd.DataFrame(errors)
print("not invertible (price ~ 0):", errors["error"].isna().sum())
print("max error on the rest:", errors["error"].max())
errors[errors["error"].isna()]
```

Cell 3.4: Compare your IV with Yahoo's IV

```
fig = go.Figure()
fig.add_scatter(x=df["iv_yahoo"], y=df["iv"], mode="markers", marker=dict(size=4), name="options")
```

```
fig.add_scatter(x=[0, 1], y=[0, 1], mode="lines", name="y = x")
fig.update_layout(title="My IV vs Yahoo IV", xaxis_title="Yahoo IV", yaxis_title="My IV")
fig.show()
```

6.3 Tasks

☐ Look at the forward table: is the implied q close to SPY's dividend yield (roughly 1–1.5%)?

☐ Find the grid points where the round-trip fails, and explain them with vega.

☐ Time the solver: `%timeit implied_vol(...)` for one option, Newton vs pure `brentq`.

☐ Explain in Markdown why your IV differs from Yahoo's (rate, dividend, mid vs last price).

☐ Commit: `"Day 3: implied forward and IV solver"`.

7 Day 4 — Smile, skew and term structure

Goal: measure and explain the shape of implied volatility across strikes (smile/skew) and across maturities (term structure).

7.1 Theory

Definition — Total implied variance

$$w(k,T) = \sigma^2_{\text{imp}}(k,T)\,T.$$

Variance of independent increments adds over time, so for fixed k the total variance must be non-decreasing in T. This is why surfaces are built and checked in (k,w) coordinates, not (K,σ).

Skew measures

- **ATM level, skew, curvature:** fit $\sigma(k) \approx a_0 + a_1 k + a_2 k^2$ on $|k| < 0.1$. ATM vol $= a_0$, skew $= a_1$, curvature $= 2a_2$.
- **25-delta risk reversal:** $RR_{25} = \sigma(\Delta_C = 0.25) - \sigma(\Delta_P = -0.25)$. Negative $\Rightarrow$ downside puts are more expensive.
- **25-delta butterfly:** $BF_{25} = \frac{1}{2}\left[\sigma(\Delta_C = 0.25) + \sigma(\Delta_P = -0.25)\right] - \sigma_{\text{ATM}}$. Measures smile curvature (tail risk).

Why equity skew exists (key interview answer)

1. Real returns have fat tails and negative skewness; the lognormal model underprices crashes.
2. Leverage effect: when the stock falls, leverage rises, so volatility rises — spot and vol are negatively correlated.
3. Demand: portfolio managers buy OTM index puts as insurance, pushing their prices up.

Term structure. ATM IV against T. In calm markets it usually slopes upward; in stress it inverts (short-dated IV above long-dated). Empirically, skew magnitude decays roughly like $T^{-1/2}$.

7.2 Code

Cell 4.1: Coordinates, vega and delta for every option

```
df["k"] = np.log(df["strike"] / df["F"])
df["w"] = df["iv"]**2 * df["T"]
greeks = bs_greeks(df["spot"], df["strike"], df["T"], r, df["q"], df["iv"], df["type"])
df["vega"], df["delta"] = greeks["vega"], greeks["delta"]
df[["expiry", "type", "strike", "k", "iv", "w", "delta", "vega"]].head()
```

Cell 4.2: Smiles: IV vs log-moneyness, one line per expiry

```
fig = go.Figure()
for exp, g in df.sort_values("k").groupby("expiry"):
    fig.add_scatter(x=g["k"], y=g["iv"], mode="lines+markers", marker=dict(size=4),
                    name=f"{exp:%Y-%m-%d}")
fig.update_layout(title=f"{TICKER} implied volatility smiles",
```

```
                  xaxis_title="log-moneyness k = ln(K/F)", yaxis_title="implied vol")
fig.write_html(f"figures/{TICKER}_smiles.html")
fig.show()
```

Cell 4.3: ATM level, skew and curvature per expiry

```
def atm_term_structure(df, band=0.1):
    rows = []
    for exp, g in df.groupby("expiry"):
        near = g[g["k"].abs() < band]
        if len(near) < 4:
            continue
        a2, a1, a0 = np.polyfit(near["k"], near["iv"], 2)
        rows.append({"expiry": exp, "T": g["T"].iloc[0], "atm_iv": a0,
                     "skew": a1, "curvature": 2 * a2})
    return pd.DataFrame(rows)

ts = atm_term_structure(df)
slope = np.polyfit(np.log(ts["T"]), np.log(ts["skew"].abs()), 1)[0]
print(f"skew ~ T^{slope:.2f}   (theory/empirics: about -0.5)")
ts
```

Cell 4.4: 25-delta risk reversal and butterfly

```
def rr_bf_25(df):
    rows = []
    for exp, g in df.groupby("expiry"):
        c = g[g["type"] == "call"].sort_values("delta")
        p = g[g["type"] == "put"].sort_values("delta")
        if len(c) < 3 or len(p) < 3:
            continue
        iv_c = np.interp(0.25, c["delta"], c["iv"])
        iv_p = np.interp(-0.25, p["delta"], p["iv"])
        gk = g.sort_values("k")
        atm = np.interp(0.0, gk["k"], gk["iv"])
        rows.append({"expiry": exp, "T": g["T"].iloc[0], "atm": atm,
                     "RR25": iv_c - iv_p, "BF25": 0.5*(iv_c + iv_p) - atm})
    return pd.DataFrame(rows)

rr = rr_bf_25(df)
rr
```

7.3 Tasks

☐ Plot ATM IV vs T (term structure) and skew vs T. Is the term structure upward or inverted today?

☐ Re-run the notebook with a single stock (AAPL). Compare: index skew is steep, single stocks are flatter and more smile-shaped. Explain in 3 sentences.

☐ Is RR_{25} negative for every expiry of SPY? What would a positive value mean?

☐ Commit: `"Day 4: smile, skew and term structure"`.

8 Day 5 — The surface: SVI fit and arbitrage checks

Goal: a smooth, continuous surface $\sigma(k, T)$ fitted to the market, plus numerical proof that it is free of static arbitrage. This is the core day of the project.

8.1 Theory

Raw SVI parameterisation (Gatheral)

For one expiry, in total variance:

$$w(k) = a + b\left(\rho\,(k-m) + \sqrt{(k-m)^2 + s^2}\right).$$

a: level b: wing slope ρ: asymmetry (skew) m: horizontal shift s: ATM curvature. Constraints: $b \geq 0$, $|\rho| < 1$, $s > 0$, and $a + b\,s\sqrt{1-\rho^2} \geq 0$ (the minimum of w, so $w \geq 0$). For large $|k|$, w grows linearly with slope $b(1 \pm \rho)$, which must be at most 2 (Roger Lee's moment formula).

Fitting. For each expiry, minimise the vega-weighted squared error (liquid ATM points count more):

$$\min_{a,b,\rho,m,s} \sum_i \text{vega}_i \left(w_{\text{SVI}}(k_i) - w_i^{\text{mkt}}\right)^2.$$

Static arbitrage conditions

1. Calendar: for each fixed k, $\partial w / \partial T \geq 0$.
2. Butterfly: the implied density must be non-negative. For a slice $w(k)$ this is equivalent to $g(k) \geq 0$ for all k, where

$$g(k) = \left(1 - \frac{k\,w'(k)}{2w(k)}\right)^2 - \frac{w'(k)^2}{4}\left(\frac{1}{w(k)} + \frac{1}{4}\right) + \frac{w''(k)}{2}.$$

Interpolation in T. Between fitted expiries $T_1 < T < T_2$, interpolate *linearly in total variance* at fixed k. If the slices are calendar-ordered, every interpolated slice is too.

8.2 Code

Cell 5.1: SVI function and fit for one slice

```
def svi_w(k, a, b, rho, m, s):
    return a + b * (rho * (k - m) + np.sqrt((k - m)**2 + s**2))

def fit_svi(k, w, weights=None):
    k, w = np.asarray(k, float), np.asarray(w, float)
    wt = np.ones_like(w) if weights is None else np.asarray(weights, float)
    wt = wt / wt.sum()
    scale = w.mean()                                  # fit in relative units: better conditioning
    loss = lambda p: np.sum(wt * ((svi_w(k, *p) - w) / scale)**2)
    cons = [{"type": "ineq", "fun": lambda p: p[0] + p[1]*p[4]*np.sqrt(1 - p[2]**2)},
            {"type": "ineq", "fun": lambda p: 2 - p[1]*(1 + abs(p[2]))}]   # Lee bound
    bounds = [(-0.5, 0.5), (1e-5, 2.0), (-0.999, 0.999), (-1.0, 1.0), (1e-4, 2.0)]
    b0 = max((w.max() - w.min()) / (k.max() - k.min()), 1e-3)
    best = None
    for rho0 in [-0.7, -0.3, 0.2]:                    # several starts, keep the best
```

```
        for s0 in [0.02, 0.1, 0.3]:
            x0 = [w.min() * 0.5, b0, rho0, 0.0, s0]
            res = minimize(loss, x0, bounds=bounds, constraints=cons, method="SLSQP",
                           options={"ftol": 1e-12, "maxiter": 500})
            if best is None or res.fun < best.fun:
                best = res
    return best.x
```

Cell 5.2: Butterfly function $g(k)$ by finite differences

```
def butterfly_g(k, params, h=1e-4):
    w = svi_w(k, *params)
    w1 = (svi_w(k + h, *params) - svi_w(k - h, *params)) / (2*h)
    w2 = (svi_w(k + h, *params) - 2*w + svi_w(k - h, *params)) / h**2
    return (1 - k*w1/(2*w))**2 - (w1**2/4) * (1/w + 0.25) + w2/2
```

Cell 5.3: The surface object

```
class VolSurface:
    def __init__(self, df):
        self.slices = []
        for exp, g in df.groupby("expiry"):
            if len(g) < 6:                                  # need enough points for 5 params
                continue
            params = fit_svi(g["k"], g["w"], g["vega"])
            self.slices.append({"expiry": exp, "T": float(g["T"].iloc[0]),
                                "F": float(g["F"].iloc[0]), "params": params})
        self.slices.sort(key=lambda s: s["T"])
        self.Ts = np.array([s["T"] for s in self.slices])

    def total_variance(self, k, T):
        k = np.atleast_1d(np.asarray(k, float))
        W = np.array([svi_w(k, *s["params"]) for s in self.slices])   # (n_T, n_k)
        return np.array([np.interp(T, self.Ts, W[:, j]) for j in range(len(k))])

    def iv(self, k, T):
        return np.sqrt(self.total_variance(k, T) / T)

    def calendar_violations(self, k_grid=np.linspace(-0.4, 0.4, 81)):
        W = np.array([svi_w(k_grid, *s["params"]) for s in self.slices])
        return int((np.diff(W, axis=0) < -1e-8).sum())

    def butterfly_violations(self, k_grid=np.linspace(-0.4, 0.4, 161)):
        return {f"{s['expiry']:%Y-%m-%d}": int((butterfly_g(k_grid, s["params"]) < 0).sum())
                for s in self.slices}

surf = VolSurface(df)
print("calendar violations:", surf.calendar_violations())
print("butterfly violations per slice:", surf.butterfly_violations())
```

Outside $[T_{\min}, T_{\max}]$ the `np.interp` call holds total variance flat, so the surface is only reliable inside the fitted maturity range.

Cell 5.4: Fit quality: market points vs SVI curve, and RMSE in vol points

```
rows = []
fig = go.Figure()
for s in surf.slices:
    g = df[df["expiry"] == s["expiry"]].sort_values("k")
    fit_iv = np.sqrt(svi_w(g["k"], *s["params"]) / s["T"])
    rows.append({"expiry": s["expiry"], "rmse_vol_pts": 100*np.sqrt(np.mean((fit_iv - g["iv"])**2))})
    fig.add_scatter(x=g["k"], y=g["iv"], mode="markers", marker=dict(size=4),
                    name=f"mkt {s['expiry']:%m-%d}")
    fig.add_scatter(x=g["k"], y=fit_iv, mode="lines", name=f"SVI {s['expiry']:%m-%d}")
fig.update_layout(title="SVI fit per expiry", xaxis_title="k", yaxis_title="IV")
```

```
fig.show()
pd.DataFrame(rows)
```

Cell 5.5: 3D surface plot

```
k_grid = np.linspace(-0.3, 0.3, 60)
T_grid = np.linspace(surf.Ts.min(), surf.Ts.max(), 40)
Z = np.array([surf.iv(k_grid, T_) for T_ in T_grid])
fig = go.Figure(go.Surface(x=k_grid, y=T_grid, z=Z, colorscale="Viridis"))
fig.update_layout(title=f"{TICKER} implied volatility surface (SVI)",
                  scene=dict(xaxis_title="k = ln(K/F)", yaxis_title="T (years)", zaxis_title="IV"))
fig.write_html(f"figures/{TICKER}_surface.html")
fig.show()
```

Cell 5.6: Export for the Streamlit app (Day 7)

```
svi_out = [{"expiry": f"{s['expiry']:%Y-%m-%d}", "T": s["T"], "F": s["F"],
            "params": [float(x) for x in s["params"]]} for s in surf.slices]
with open(f"data/{TICKER}_svi.json", "w") as fh:
    json.dump({"ticker": TICKER, "spot": float(S0), "r": r, "slices": svi_out}, fh, indent=2)
df.to_parquet(f"data/{TICKER}_iv_points.parquet")
ts.to_csv(f"data/{TICKER}_term_structure.csv", index=False)
```

8.3 Tasks

☐ SVI RMSE below about 1 vol point for liquid expiries; investigate any slice that is worse.

☐ Plot $g(k)$ for each slice; any region below 0 is butterfly arbitrage.

☐ If calendar violations > 0: refit slices in order of T, adding a penalty $\lambda \sum_k \max(0, w_{T-1}(k) - w_T(k))^2$ to the loss.

☐ Naive comparison: interpolate raw IV with `griddata((df["k"], df["T"]), df["iv"], …)` and plot it. Show the bumps and explain why SVI is better.

☐ Commit: `"Day 5: SVI surface and arbitrage checks"`.

9 Day 6 — Options analysis with the surface

Goal: use the surface to answer real questions — what distribution does the market imply, is volatility overpriced, and how do common strategies behave.

9.1 Theory

Theorem — Breeden–Litzenberger (1978)

Since $C(K) = e^{-rT}\int_0^\infty (x-K)^+ q_T(x)\,dx$, differentiating twice in K gives

$$\frac{\partial C}{\partial K} = -e^{-rT}\,\mathbb{Q}(S_T > K), \qquad q_T(K) = e^{rT}\,\frac{\partial^2 C(K,T)}{\partial K^2}.$$

Use $C(K)$ from the *smooth SVI surface*, never raw quotes — second differences amplify noise. Checks: $\int q\,dK \approx 1$, $\int K\,q\,dK \approx F$, $q \geq 0$.

Definition — Volatility risk premium

$$\sigma_{\text{real}} = \sqrt{252}\ \text{std}\left(\ln\frac{S_t}{S_{t-1}}\right), \qquad \text{VRP} = \sigma^{30\text{d}}_{\text{ATM}} - \sigma^{30\text{d}}_{\text{real}}.$$

For indices VRP is usually positive: option sellers are paid for bearing crash risk.

Strategies (each leg priced at its own surface vol):

Strategy	Position	View traded
Straddle	long ATM call + long ATM put	volatility level (long vega, long gamma)
Risk reversal	long OTM call + short OTM put	skew and direction
Butterfly	long K_1, short $2\times K_2$, long K_3 calls	smile curvature, pinned price
Calendar	short near ATM, long far ATM	term structure

9.2 Code

Cell 6.1: Risk-neutral density from the surface

```
def risk_neutral_density(surf, slice_idx, r, n=800, n_sd=(-6, 5)):
    s = surf.slices[slice_idx]
    T, F = s["T"], s["F"]
    sd = np.sqrt(svi_w(0.0, *s["params"]))             # ATM std-dev of ln(S_T): grid scales with T
    K = F * np.exp(np.linspace(n_sd[0]*sd, n_sd[1]*sd, n))
    sigma = np.sqrt(svi_w(np.log(K / F), *s["params"]) / T)
    C = bs_price(F * np.exp(-r*T), K, T, r, 0.0, sigma, "call")   # Black-76 via forward
    dens = np.exp(r*T) * np.gradient(np.gradient(C, K), K)
    dK = np.gradient(K)
    return K, dens, {"T": T, "F": F, "mass": float(np.sum(dens*dK)),
                     "mean": float(np.sum(K*dens*dK))}

def lognormal_density(K, F, T, sigma):
    d2 = (np.log(F / K) - 0.5*sigma**2*T) / (sigma*np.sqrt(T))
    return norm.pdf(d2) / (K * sigma * np.sqrt(T))

fig = go.Figure()
for i in [0, len(surf.slices)//2, len(surf.slices) - 1]:
```

```
    K, dens, info = risk_neutral_density(surf, i, r)
    atm = float(np.sqrt(svi_w(0.0, *surf.slices[i]["params"]) / info["T"]))
    fig.add_scatter(x=K, y=dens, name=f"market T={info['T']:.2f}")
    fig.add_scatter(x=K, y=lognormal_density(K, info["F"], info["T"], atm),
                    name=f"lognormal T={info['T']:.2f}", line=dict(dash="dot"))
    print(f"T={info['T']:.3f}  mass={info['mass']:.4f}  mean={info['mean']:.2f}  F={info['F']:.2f}")
fig.update_layout(title="Risk-neutral density: market vs lognormal", xaxis_title="S_T")
fig.show()
```

Cell 6.2: Crash probability: market vs lognormal

```
i = len(surf.slices) // 2
K, dens, info = risk_neutral_density(surf, i, r)
barrier = 0.9 * S0
p_mkt = np.sum((dens * np.gradient(K))[K < barrier])
atm = float(np.sqrt(svi_w(0.0, *surf.slices[i]["params"]) / info["T"]))
d2 = (np.log(info["F"] / barrier) - 0.5*atm**2*info["T"]) / (atm*np.sqrt(info["T"]))
print(f"P(S_T < 0.9 S0), T={info['T']:.2f}:  market {p_mkt:.3%}   lognormal {norm.cdf(-d2):.3%}")
```

Cell 6.3: Volatility risk premium

```
hist = yf.Ticker(TICKER).history(period="1y")["Close"]
rv30 = float(np.log(hist).diff().dropna().tail(21).std() * np.sqrt(252))
iv30 = float(surf.iv(0.0, 30/365)[0])
print(f"30d ATM implied {iv30:.2%}   30d realized {rv30:.2%}   VRP {iv30 - rv30:+.2%}")
```

21 trading days ≈ 30 calendar days. If the shortest fitted expiry is longer than 30 days, `surf.iv` holds variance flat and the 30-day number is approximate.

Cell 6.4: Strategy value and Greeks from the surface

```
def strategy(legs, surf, S, r, T, q=0.0):
    """legs = list of (quantity, strike, 'call'/'put'), all with maturity T."""
    F = S * np.exp((r - q) * T)
    out = dict.fromkeys(["price", "delta", "gamma", "vega", "theta"], 0.0)
    for qty, K, kind in legs:
        sig = float(surf.iv(np.log(K / F), T)[0])
        out["price"] += qty * float(bs_price(S, K, T, r, q, sig, kind))
        g = bs_greeks(S, K, T, r, q, sig, kind)
        for key in ["delta", "gamma", "vega", "theta"]:
            out[key] += qty * float(g[key])
    return out

T_mid = surf.slices[len(surf.slices)//2]["T"]
q_mid = float(df["q"].median())
K_atm = round(S0)
book = {
    "straddle":      [(1, K_atm, "call"), (1, K_atm, "put")],
    "risk reversal": [(1, round(1.05*S0), "call"), (-1, round(0.95*S0), "put")],
    "butterfly":     [(1, round(0.95*S0), "call"), (-2, K_atm, "call"), (1, round(1.05*S0), "call")],
}
pd.DataFrame({name: strategy(legs, surf, S0, r, T_mid, q_mid) for name, legs in book.items()}).T
```

Cell 6.5: Payoff at expiry vs value today (straddle)

```
S_range = np.linspace(0.8*S0, 1.2*S0, 200)
legs = book["straddle"]
payoff = sum(q_ * np.maximum(S_range - K_, 0) if kd == "call" else q_ * np.maximum(K_ - S_range, 0)
             for q_, K_, kd in legs)
value = [strategy(legs, surf, s_, r, T_mid, q_mid)["price"] for s_ in S_range]
fig = go.Figure()
fig.add_scatter(x=S_range, y=payoff, name="payoff at expiry")
fig.add_scatter(x=S_range, y=value, name="value today")
fig.update_layout(title="Straddle", xaxis_title="Spot")
fig.show()
```

9.3 Tasks

- ☐ Density checks: mass ≈ 1 and mean $\approx F$ for every slice; the market density has a fatter left tail than the lognormal.
- ☐ Explain the gap between market and lognormal crash probability in 2–3 sentences.
- ☐ Record the VRP; is it positive today?
- ☐ Heatmap of Gamma and Vega over a (K, T) grid using surface vols — where is risk concentrated?
- ☐ Save key figures to `figures/` for the README. Commit: `"Day 6: density, VRP, strategies"`.

10 Day 7 — Streamlit app, README and GitHub

Goal: a live web app anyone can open, and a repo a recruiter understands in 60 seconds.

Design. The app does *not* recompute anything. It reads the files your notebook exported on Day 5 (`data/<TICKER>_svi.json`, `_iv_points.parquet`, `_term_structure.csv`). So: one notebook does the work, and `app.py` is a small viewer. Run the notebook for SPY and AAPL and the app shows both. It also deploys reliably, because the cloud server never calls yfinance.

Cell 7.1: `app.py` — the only .py file in the repo

```
import glob, json
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import streamlit as st

st.set_page_config(page_title="IV Surface", layout="wide")
st.title("Implied Volatility Surface & Options Analysis")

def svi_w(k, a, b, rho, m, s):
    return a + b * (rho * (k - m) + np.sqrt((k - m)**2 + s**2))

tickers = sorted(f.split("/")[-1].replace("_svi.json", "") for f in glob.glob("data/*_svi.json"))
ticker = st.sidebar.selectbox("Ticker", tickers)

@st.cache_data
def load(ticker):
    with open(f"data/{ticker}_svi.json") as fh:
        svi = json.load(fh)
    pts = pd.read_parquet(f"data/{ticker}_iv_points.parquet")
    ts = pd.read_csv(f"data/{ticker}_term_structure.csv")
    return svi, pts, ts

svi, pts, ts = load(ticker)
slices = svi["slices"]
Ts = np.array([s["T"] for s in slices])

def surface_iv(k, T):
    W = np.array([svi_w(k, *s["params"]) for s in slices])
    return np.sqrt(np.array([np.interp(T, Ts, W[:, j]) for j in range(len(k))]) / T)

c1, c2, c3 = st.columns(3)
c1.metric("Spot", f"{svi['spot']:.2f}")
c2.metric("Risk-free rate", f"{svi['r']:.2%}")
c3.metric("Expiries fitted", len(slices))

tab1, tab2, tab3 = st.tabs(["3D surface", "Smiles", "Term structure"])
with tab1:
    k = np.linspace(-0.3, 0.3, 60)
    T_grid = np.linspace(Ts.min(), Ts.max(), 40)
    Z = np.array([surface_iv(k, T_) for T_ in T_grid])
    fig = go.Figure(go.Surface(x=k, y=T_grid, z=Z, colorscale="Viridis"))
    fig.update_layout(height=650, scene=dict(xaxis_title="k = ln(K/F)",
                      yaxis_title="T (years)", zaxis_title="IV"))
    st.plotly_chart(fig, width="stretch")
with tab2:
    labels = [s["expiry"] for s in slices]
    choice = st.selectbox("Expiry", labels)
    s = slices[labels.index(choice)]
    g = pts[pts["expiry"].dt.strftime("%Y-%m-%d") == choice].sort_values("k")
    fig = go.Figure()
    fig.add_scatter(x=g["k"], y=g["iv"], mode="markers", name="market")
```

```
    fig.add_scatter(x=g["k"], y=np.sqrt(svi_w(g["k"], *s["params"]) / s["T"]), name="SVI")
    fig.update_layout(xaxis_title="k", yaxis_title="IV")
    st.plotly_chart(fig, width="stretch")
with tab3:
    st.line_chart(ts.set_index("T")[["atm_iv"]])
    st.dataframe(ts)
```

Run locally: `streamlit run app.py`. Once it works, add a fourth tab for the Day 6 density (export it from the notebook the same way).

10.1 Deploy

1. Push the repo to GitHub (public), *including* the `data/` exports.
2. Go to Streamlit Community Cloud, sign in with GitHub, choose the repo and `app.py`.
3. Put the app link at the top of the README.

10.2 README checklist

☐ One-line summary, live app link, a screenshot of the 3D surface.

☐ Problem and motivation (Section 1 of this guide, in your own words).

☐ Math summary: BS formula, IV definition and uniqueness, SVI, arbitrage conditions, Breeden–Litzenberger.

☐ Results: SVI RMSE, arbitrage violations, VRP, density plot, skew observations (SPY vs AAPL).

☐ How to run: `pip install -r requirements.txt`, open the notebook, `streamlit run app.py`.

☐ Limitations: delayed yfinance data; SPY options are American but priced as European (small early-exercise error, mostly ITM puts — which you removed); flat risk-free rate.

10.3 Final push

Terminal

```
jupyter nbconvert --to notebook --execute --inplace iv_surface.ipynb   # proves it runs top to bottom
git add .
git commit -m "Day 7: Streamlit app and README"
git branch -M main
git remote add origin https://github.com/<your-new-username>/iv-surface.git
git push -u origin main
```

11 Stretch goals and further reading

Extension	What you add	Why it matters
Daily snapshots	Save the surface every day	Study how skew and VRP move over time
Dupire local vol	$\sigma_{\text{loc}}(K,T)$ from the surface	Prices exotics consistently with vanillas
Heston calibration	Fit $(\kappa,\theta,\xi,\rho,v_0)$ with characteristic-function pricing	Stochastic vol model; compare its smile to SVI
SSVI	One global parameterisation over all T	Arbitrage-free by construction
Delta-hedging backtest	Sell a straddle, hedge daily, track P&L	Shows how VRP is earned and gamma risk is paid
RL hedging	Agent learns hedge ratios with transaction costs	Links to your reinforcement learning study

Reading

Listed from memory — please double-check titles and editions.

- J. Gatheral, *The Volatility Surface: A Practitioner's Guide* — SVI, local vol, Heston, skew.
- J. Hull, *Options, Futures, and Other Derivatives* — Black–Scholes, Greeks, IV basics.
- S. Natenberg, *Option Volatility and Pricing* — the trader's view of volatility and strategies.
- J. Gatheral & A. Jacquier, "Arbitrage-free SVI volatility surfaces" (2014) — the $g(k)$ condition and SSVI.
- D. Breeden & R. Litzenberger, "Prices of state-contingent claims implicit in option prices" (1978).